ITA03 – MOBILE COMPUTING

ASSIGNMENT 4 – CO4

PART I AND PART II

STUDENT DETAILS	
Name	V. Naga Nischay
Register Number	192472255
Course	ITA0329 – Mobile Computing
Assignment	Assignment 4 – CO4

PART I – MOBILE APPLICATION DEVELOPMENT ENVIRONMENT

TASK 1 – ARCHITECTURE REVIEW

1.1 Diagnose the poor architecture description

The junior engineer's note identifies only hardware, kernel, runtime, permissions, battery saving and framework. These labels are too broad to guide implementation or evaluation. The following technical deficiencies must be corrected.

- No.: 1 | Deficiency: Missing hardware abstraction and device drivers | Why the description is insufficient: It does not explain how the OS controls display, storage, sensors, radio, camera and other hardware. An engineer cannot identify the driver or HAL boundary.
- No.: 2 | Deficiency: Kernel responsibilities are vague | Why the description is insufficient: The kernel does not simply “run applications”. It manages processes, threads, memory, scheduling, interrupts, device access and low-level security boundaries.
- No.: 3 | Deficiency: Native libraries and system services are omitted | Why the description is insufficient: Applications depend on graphics, media, database, networking and system-service libraries. Their interfaces and dependencies are not described.
- No.: 4 | Deficiency: Runtime is undefined | Why the description is insufficient: The note does not identify whether code is bytecode or native machine code, who loads it, or whether interpretation, JIT or AOT compilation occurs.
- No.: 5 | Deficiency: Permission enforcement is unspecified | Why the description is insufficient: It does not state who declares, grants, revokes or enforces permissions, or whether enforcement occurs at install time, runtime, framework API or kernel boundary.
- No.: 6 | Deficiency: Power management has no trigger or mechanism | Why the description is insufficient: “When needed” does not define idle detection, CPU frequency scaling, suspend, wakelocks, background limits or wake-up conditions.

- No.: 7 | Deficiency: Application-to-OS communication is missing | Why the description is insufficient: No IPC mechanism or system-service boundary is identified. The application call path cannot be traced.
- No.: 8 | Deficiency: No isolation or process model | Why the description is insufficient: The note omits per-app identity, sandboxing, process creation, memory isolation and lifecycle management.
- No.: 9 | Deficiency: No dependency direction | Why the description is insufficient: It does not say what each layer exposes upward or which lower layer it depends on, so the architecture cannot be evaluated.
- No.: 10 | Deficiency: Security and power claims are untestable | Why the description is insufficient: No enforcement point, policy, event, metric or observable state is named.

1.2 Repaired architecture specification – Android

Android is selected because its layered architecture clearly separates hardware drivers, Linux kernel, HAL, native libraries, ART, application framework and applications.

Layer (bottom to top)	Responsibility	Interface exposed upward	Dependency below
Hardware	CPU, memory, display, storage, sensors, camera, modem and radio.	Physical computation, input/output and connectivity.	Physical device components.
Linux kernel and device drivers	Process/thread scheduling, virtual memory, IPC primitives, networking, power interfaces, device drivers and security enforcement such as SELinux.	System calls, device nodes, scheduling and memory services.	Hardware and architecture-specific drivers.
Hardware Abstraction Layer (HAL)	Provides standard interfaces for hardware functions such as camera, audio, sensors and Bluetooth.	Stable hardware service interfaces to Android system components.	Kernel drivers and device firmware.
Native libraries and Android system services	Libraries provide graphics, media, database, SSL and C/C++ functionality; native daemons and services manage system capabilities.	Native APIs and service endpoints used by framework components.	Kernel, HAL and native runtime support.
Android Runtime (ART)	Loads DEX-based application code, verifies and executes it; uses interpretation, JIT and profile-guided AOT compilation; manages managed heap and garbage collection.	Runtime execution, class loading, memory management and JNI access.	Linux process/memory services and native libraries.
Application framework	Provides Activity Manager, Window Manager, Package Manager, Location Manager, Notification Manager and other high-level APIs.	Java/Kotlin framework APIs and Binder-based system-service access.	ART, native libraries and system services.
Applications	Implements user-facing features, activities, services, receivers and providers under app identity and lifecycle rules.	UI and application functionality to the user and other authorised apps.	Framework APIs, runtime and OS services.

Explicit architectural locations

- Kernel: Linux kernel layer; manages scheduling, memory, IPC primitives, drivers and low-level enforcement.
- Runtime: ART layer; loads DEX, verifies code, interprets/JIT-compiles and uses profile-guided AOT compilation.
- Security enforcement: application UID sandbox, SELinux in the kernel, permission checks in framework/system services, package signing and Android Keystore for protected keys.
- Power management: kernel CPU frequency and suspend mechanisms coordinated with Android Power HAL, Power Manager, Doze, App Standby and application wake-lock policy.
- Application-to-OS path: app calls framework API → framework service → Binder IPC → native/system service → HAL or kernel driver → hardware.

1.3 Design defence – under 100 words

The repaired description is an architectural specification because it names every major layer, assigns each layer a bounded responsibility, identifies the interface exposed upward and states the dependency below. It locates the Linux kernel, ART runtime, permission and sandbox enforcement points, and power-management mechanisms explicitly. It also defines the application-to-system-service communication path through framework APIs and Binder IPC. These details allow another engineer to trace a request, identify the responsible component and test its behaviour. Unlike a marketing overview, the specification makes technical boundaries and dependencies observable.

TASK 2 – ANDROID VS iOS VS WINDOWS

2.1 Requirement model

- Element: Input | Definition for secure background messaging: Platform architecture, kernel family, runtime, app lifecycle, IPC, security model and background-execution policy.
- Element: Output | Definition for secure background messaging: A platform comparison and an implementation decision for reliable message receipt.
- Element: Objective | Definition for secure background messaging: Deliver incoming messages securely while the screen is off, with acceptable delay and battery consumption.
- Element: Constraints | Definition for secure background messaging: Background execution restrictions, network availability, notification permission, encryption, battery limits, process termination and OS version.
- Element: Decision | Definition for secure background messaging: Select platform-specific delivery and notification architecture, not one identical background loop for all platforms.
- Element: Information required | Definition for secure background messaging: Push service, process model, IPC, app sandbox, runtime, notification service, background task rules and secure storage.

2.2 Hypothesis

The background-execution and process-lifecycle policy is expected to affect message delivery more than the kernel family alone, because the OS may suspend or terminate an application while relying on a privileged push service to wake or notify it.

2.3 Architectural comparison matrix

Dimension	Android	iOS	Windows 8 / WinRT	Messaging consequence
Kernel family	Linux kernel; modular monolithic kernel with loadable modules and device drivers.	Darwin/XNU; hybrid kernel combining Mach and BSD components.	Windows NT kernel; hybrid architecture with executive services and kernel-mode components.	All provide process, memory and device services, but their user-mode APIs and background policies differ.
Kernel programming interface	Linux system calls and Android-specific native interfaces; apps normally use framework APIs rather than direct kernel calls.	Darwin/Mach and BSD interfaces are mediated by public frameworks and entitlements.	Win32/NT and WinRT APIs expose OS services through documented interfaces.	Direct kernel programming is not a portable application strategy.
Runtime	ART executes DEX-based code with interpretation, JIT and profile-guided AOT.	Objective-C runtime supports dynamic dispatch, selectors and runtime metadata; Swift uses native compiled code and ARC.	WinRT is a Windows application API/runtime model; language projections support native and managed languages.	Runtime affects startup, memory and native API access, but background delivery is also controlled by OS policy.
Application framework	Android SDK: Activity, Service, BroadcastReceiver, ContentProvider and system managers.	Cocoa Touch/UIKit: UIApplication, UIViewController, notification and background-task APIs.	WinRT APIs, app lifecycle and background task infrastructure.	Each platform requires lifecycle-aware message handling.
Isolation	Per-app Linux UID, sandbox, SELinux and permission controls.	Per-app sandbox, code signing, entitlements and data containers.	AppContainer isolation, capabilities and package identity.	Sensitive message data must remain inside app-controlled storage.
IPC	Binder is the principal Android framework IPC mechanism; intents and content providers offer higher-level interactions.	Mach ports and XPC support interprocess communication.	App services and Windows IPC mechanisms support component communication.	System push/notification services communicate with apps through OS-defined channels.

2.4 Incoming message desk-check – Android and iOS

- Step: 1. Network event | Android path: Push message reaches Google/approved push infrastructure and device network stack. | iOS path: APNs delivers the push notification to the device.
- Step: 2. Kernel / radio | Android path: Modem and network driver deliver network data to system networking components. | iOS path: Baseband/network stack and Darwin networking deliver the push to system services.
- Step: 3. System service | Android path: Google Play services / Android push component processes the push and coordinates app delivery or notification. | iOS path: iOS push service receives the notification and applies system policy.
- Step: 4. App wake | Android path: If permitted, app process may be started or briefly awakened for supported background handling; otherwise system displays notification. | iOS path: System may display alert; silent background execution is limited and not guaranteed.
- Step: 5. Runtime | Android path: ART loads or resumes the application process and executes callback code. | iOS path: Objective-C/Swift runtime executes permitted notification/background callback.
- Step: 6. Application | Android path: App validates message, stores it securely and updates notification/UI when allowed. | iOS path: App processes permitted payload, stores data and updates notification/UI.

The key distinction is that push delivery is mediated by privileged OS services. An application should not keep its own permanent network loop alive merely to receive messages.

2.5 Changed condition – application terminated

Prediction: Android may recreate an eligible process when a supported push message arrives, subject to force-stop state, vendor restrictions and message priority. iOS may display a visible notification without launching the application; silent background execution is discretionary and constrained. The responsible elements are the push service, app lifecycle manager and background execution policy.

- Platform: Android | Relevant trace after termination: Push service receives message → OS decides whether app may be started → process/ART created if eligible → callback or notification handling. | Expected result: Eligible push can trigger limited app work; force-stopped apps and restricted cases behave differently.
- Platform: iOS | Relevant trace after termination: APNs delivers push → system decides alert/background handling → visible alert may be shown; app callback only if background execution is permitted. | Expected result: Visible notification can appear without the app process running; silent execution is not guaranteed.

The prediction must be checked against a real device and OS version. The assignment does not provide actual test logs, so no observed outcome is claimed here.

TASK 3 – RUNTIME ENVIRONMENT ANALYSIS

3.1 Execution model

- Platform: Android / Dalvik (legacy) | Delivered application form: DEX bytecode in APK. | Loader: Android package manager and Dalvik VM process loader. | When executable instructions are available: Bytecode is verified/loaded and interpreted or JIT-compiled at runtime; some optimisation may be performed ahead of execution.
- Platform: Android / ART (current) | Delivered application form: DEX bytecode plus profile/compiled artifacts as available. | Loader: Zygote-created app process and ART class/runtime loader. | When executable instructions are available: Interpretation/JIT at runtime and profile-guided AOT compilation; exact mode depends on profile and compilation state.
- Platform: iOS / Objective-C runtime | Delivered application form: Signed native Mach-O executable and libraries. | Loader: launchd and dyld dynamic loader. | When executable instructions are available: Native machine code is produced before execution by the build toolchain; runtime performs dynamic linking and Objective-C dispatch.
- Platform: Windows / WinRT | Delivered application form: Native PE binaries or managed assemblies depending on language/toolchain. | Loader: Windows process activation and relevant runtime loader. | When executable instructions are available: Native code is loaded as machine code; managed components execute through their runtime/compiler model.

3.2 Android application launch sequence

- Order: 1 | Action and purpose: User taps icon; launcher resolves package and launch activity. | Component: Launcher / Package Manager | Prerequisite: Installed, valid package and launcher activity.
- Order: 2 | Action and purpose: System checks app process; creates one if absent. | Component: Activity Manager / Zygote | Prerequisite: System services and Zygote available.
- Order: 3 | Action and purpose: Zygote forks application process and establishes app UID/security context. | Component: Zygote / Linux kernel | Prerequisite: Zygote running; kernel process and memory services available.
- Order: 4 | Action and purpose: ART initialises runtime, loads classes and verifies DEX as needed. | Component: ART | Prerequisite: App process and runtime libraries available.
- Order: 5 | Action and purpose: Application object and process components are created; dependency injection/configuration may run. | Component: Application framework | Prerequisite: ART and framework services initialised.
- Order: 6 | Action and purpose: Activity lifecycle callbacks execute: onCreate, onStart and onResume. | Component: Activity Manager / app main thread | Prerequisite: Activity instance and window services available.
- Order: 7 | Action and purpose: Layout is measured, drawn and submitted to rendering pipeline. | Component: View system / RenderThread / SurfaceFlinger | Prerequisite: Window, graphics libraries and display services available.
- Order: 8 | Action and purpose: First frame is presented. | Component: Display compositor / hardware | Prerequisite: Rendered buffer and display pipeline available.

Each step depends on lower layers. The kernel must create and schedule the process before ART can initialise. ART must load application classes before framework callbacks can invoke them. Window and graphics services must be ready before the first screen can be presented.

3.3 Runtime strategy comparison

Aspect	Dalvik (legacy Android)	Objective-C runtime / iOS	Windows runtime model
Execution	DEX bytecode interpreted and JIT-compiled during execution.	Native compiled code; Objective-C runtime provides dynamic method dispatch and metadata.	WinRT is an API/runtime contract; execution depends on native or managed language toolchain.
Compilation	JIT converts frequently executed bytecode to native instructions; later ART adds AOT/profile-guided compilation.	Ahead-of-time native compilation by compiler/linker.	Native code compiled ahead of execution; managed code may use CLR compilation.
Memory management	Managed heap with garbage collection.	Objective-C objects commonly use Automatic Reference Counting (ARC); Swift also uses ARC for class instances.	Native memory management or managed garbage collection depending on language.
Responsiveness	GC pauses and allocation pressure can affect frame timing; JIT may improve hot code after warm-up.	ARC performs retain/release operations; avoids tracing GC pauses but reference cycles can leak.	Depends on runtime and language; managed GC may pause while native code has manual ownership.
Battery	JIT/GC activity and allocation rate consume CPU energy.	ARC bookkeeping and native execution; efficient when ownership is managed correctly.	Depends on runtime, allocation and scheduling model.
Portability	DEX/runtime model supports multiple hardware architectures through Android runtime.	Native binary is built for Apple target architectures and platform APIs.	WinRT API model spans supported Windows app languages, but binaries and APIs remain platform-specific.

3.4 Suitability for a low-memory device

A native ahead-of-time compiled design with predictable object ownership can reduce runtime compilation and tracing-GC pressure, so it can be attractive for a tightly constrained device. However, the best choice depends on the language, framework footprint and application allocation pattern. Dalvik's managed heap simplifies memory safety but may add GC and runtime overhead. ARC avoids a tracing collector but requires careful ownership and cycle management. Therefore, a small native application with controlled allocations is the strongest low-memory candidate; no runtime is universally smallest without measurement.

3.5 Cold-start and warm-start desk-check

- Launch stage: Process creation | Cold start: Required if process is absent. | Warm start: Skipped if process remains alive.
- Launch stage: Runtime initialisation | Cold start: Required. | Warm start: Already initialised; reused.
- Launch stage: Class loading | Cold start: Initial classes loaded as required. | Warm start: Previously loaded classes may be reused.
- Launch stage: Application object | Cold start: Created for new process. | Warm start: Existing application object reused.

- Launch stage: Activity creation | Cold start: Activity created and callbacks run. | Warm start: May resume existing Activity or create a new one depending on task state.
- Launch stage: Rendering | Cold start: First frame must be produced. | Warm start: Frame is redrawn/resumed; cached resources may reduce work.

3.6 Dependency analysis

The kernel process manager and memory manager must be available before an application process can exist. ART or the relevant runtime must initialise before managed classes and application callbacks execute. Framework services must be available before Activity lifecycle callbacks can request windows, permissions or notifications. Graphics and compositor services must be ready before a frame can reach the display. This dependency order prevents higher layers from calling interfaces that have not been created.

TASK 4 – SECURITY FORENSICS: SECURENOTES

4.1 Identify the defects

- Original decision: Request every permission at installation for future use. | Security mechanism bypassed: Least-privilege permission model and runtime consent. | Risk and reason: Excessive access increases attack surface and weakens user control. Permissions should be requested only when a feature needs them.
- Original decision: Save login token in a file readable by other apps. | Security mechanism bypassed: Application sandbox and secure storage. | Risk and reason: Another app or compromised environment may read the token, enabling account impersonation.
- Original decision: Accept updates from any source without producer verification. | Security mechanism bypassed: Package/code signing and trusted update chain. | Risk and reason: A malicious package can replace trusted application code or inject malicious functionality.
- Original decision: Use plain unencrypted server connections. | Security mechanism bypassed: TLS transport protection and server authentication. | Risk and reason: An attacker on the network can read or modify credentials and notes; confidentiality and integrity are lost.

4.2 Attack scenario

Scenario: a user connects to a malicious public Wi-Fi hotspot. A malicious application already installed on the device attempts to read the shared token file. At the same time, the attacker intercepts SecureNotes traffic and serves a modified update package.

1. The user logs in; SecureNotes writes the token to a file that is readable outside its sandbox.
2. The malicious application accesses the shared file and copies the login token. Confidentiality is first lost at this step.
3. The attacker uses the token to authenticate to the service and retrieve or alter notes.
4. The user checks for an update over an unencrypted connection.
5. The network attacker intercepts the update response and substitutes a malicious package because the application does not verify the producer's signature.
6. The malicious update executes with the application's granted privileges and can continue stealing data. Integrity is lost when the untrusted update is accepted.

4.3 Minimal repairs for Android

Defect	**Repair**	**Android mechanism and architecture location**
Excessive permissions	Declare only required permissions and request dangerous permissions in context when the user invokes the relevant feature.	Package manifest and framework runtime permission controller; permission checks at framework/service boundary.
Shared token file	Store token in app-private storage and encrypt sensitive values using a key protected by Android Keystore.	Linux UID sandbox, SELinux and Keystore-backed cryptographic key service.
Unverified updates	Accept updates only through trusted distribution/update channel and verify package signature continuity.	Package Manager signature verification and trusted installer/update process.
Plain HTTP	Use HTTPS with TLS certificate validation; consider certificate pinning where operationally	Network Security Configuration / TLS provider and application networking layer.

	justified and maintain rotation strategy.	

4.4 Regression check

Legitimate behaviour: SecureNotes reads the user's contacts only after the user selects “Import contact” and grants the relevant contact permission. The app declares the required permission, requests it at the point of use and handles denial without crashing. The token remains in app-private encrypted storage, updates remain signature-verified and server traffic remains protected by TLS. Therefore, the security repairs do not remove the user's consent-based contact-import feature.

4.5 Security verdict

The original design is insecure and should not be released. The minimum release conditions are least-privilege permissions, app-private and encrypted token storage, trusted signed updates and authenticated TLS communication. Residual risks include rooted devices, compromised OS components, malicious accessibility services, compromised server credentials and vulnerabilities in trusted third-party libraries. These risks require monitoring, secure development and incident response.

TASK 5 – POWER MANAGEMENT REASONING

5.1 Power-state model

- State: ACTIVE | Hardware and kernel behaviour: Display and input active; CPU scheduled normally; sensors sampled at configured rate; radio available as needed. | Entry condition: User interaction, active workout, urgent event or foreground tracking session. | Exit condition: Screen timeout, user lock or inactivity. | Permitted work: Foreground UI, sensor sampling, short processing and scheduled network work.
- State: IDLE | Hardware and kernel behaviour: Display off or dim; CPU frequency reduced when load is low; background execution restricted; timers and sensors may be batched. | Entry condition: No interaction for configured interval and no foreground-critical work. | Exit condition: User interaction, scheduled job, push event or motion trigger. | Permitted work: Short scheduled jobs, permitted background service work and batched sensor events.
- State: SUSPENDED / DEEP IDLE | Hardware and kernel behaviour: CPU cores enter low-power state; most app execution paused; only wake-capable sources and system maintenance events remain. | Entry condition: No active wakelock or immediate work; system enters suspend/Doze conditions. | Exit condition: Alarm, push/system wake event, user interaction or hardware wake source. | Permitted work: Minimal system wake-up processing; app work only when explicitly permitted.

5.2 Urgent push notifications while suspended

The unsafe assumption is that the fitness application must keep the processor awake permanently to avoid missing every sensor event. This prevents deep idle and drains the battery. Urgent notifications should instead use the platform push service, which can wake the device for a bounded period. Sensor data should be batched and timestamped. A partial wakelock should be held only for a short, justified operation and released in all success and failure paths. The application must stop maintaining a permanent CPU wake lock and continuous polling loop.

5.3 Battery saver at 15%

- Layer: Kernel / hardware | Policy change at 15% battery: Lower CPU frequency and voltage through DVFS; prefer efficiency cores; reduce non-essential wakeups; allow deeper idle states.
- Layer: Runtime / OS services | Policy change at 15% battery: Restrict background execution, defer non-urgent jobs, batch alarms and network work, reduce scheduling frequency.
- Layer: Application | Policy change at 15% battery: Reduce sensor sampling when stationary, batch uploads, lower UI animation and screen update rate, pause non-essential analytics and retain urgent event handling.
- Layer: User-visible trade-off | Policy change at 15% battery: Workout route and statistics may update less frequently when stationary or under low battery. This is acceptable because critical emergency and user-requested events remain prioritised while the device gains additional operating time.

Battery saver must not silently discard collected sensor data. Records are timestamped and queued locally; upload is deferred only when the application can safely retain the data.

TASK 6 – RURALCARE FIELD-DATA COLLECTION SYSTEM

6.1 Stage 1 – Problem interpretation

- Element: Input | Definition: Three device classes, patient data, field-worker actions, battery state, connectivity, available storage and security capabilities.
- Element: Output | Definition: A platform-specific data capture, secure offline storage and reliable synchronisation system.
- Element: Objective | Definition: Complete a ten-hour field shift, protect patient information and synchronise every record without loss or duplication.
- Element: Constraints | Definition: Class A Android smartphone, Class B iPhone, Class C J2ME/JME or WML feature phone; offline for hours; limited battery and storage; lost-device risk.
- Element: Decisions | Definition: Native or web client per class, local data format, encryption method, sync protocol, retry policy and retention rules.

6.2 Stage 2 – Platform strategy

Use native applications for Android and iOS because they provide stronger access to secure storage, OS permission controls, background scheduling, encryption and platform lifecycle management. Use a constrained J2ME client for Class C devices that support Java ME, with encrypted or minimised local records where the handset permits. For WML-only devices, provide a low-data server-rendered workflow and avoid storing sensitive patient details locally. If a feature phone has no trustworthy secure storage, restrict it to a minimum-data mode and require server-side retrieval when online.

Class	Strategy	Reason
A – Android	Native Android app using Kotlin/Java, Room/SQLite, Android Keystore, WorkManager and TLS.	Strong offline database, hardware-backed key options, runtime permissions and background work controls.
B – iPhone	Native iOS app using Swift, Core Data/SQLite, Keychain, Data Protection and background URLSession where permitted.	Keychain and file protection support secure local records; OS-controlled background execution.
C – J2ME/JME	Small Java ME client with RMS/local record queue and application-level encryption only where supported by device cryptography.	Legacy runtime and storage are constrained; minimise data and sync in small batches.
C – WML	Server-rendered WML forms over secure transport where available; keep sensitive records server-side and use minimal temporary session data.	WML browser may have limited storage and security capabilities; do not assume secure offline database.

6.3 Stage 3 – Constraint table

- Class: Android | Runtime: ART; Kotlin/Java runtime. | Security mechanisms: UID sandbox, SELinux, runtime permissions, Keystore, file-based encryption, TLS. | Power controls: Doze, App Standby, WorkManager, foreground service for justified ongoing work. | Offline storage: Encrypted Room/SQLite database; app-private files; Keystore-protected key. | Connectivity: Wi-Fi, cellular, offline queue and retry.
- Class: iOS | Runtime: Native Swift/Objective-C runtime; ARC. | Security mechanisms: App sandbox, code signing, entitlements, Keychain, Data Protection, TLS. | Power controls: Background App Refresh,

BGTaskScheduler, background URLSession, system suspension. | Offline storage: Protected files, Core Data/SQLite with file protection, Keychain keys. | Connectivity: Wi-Fi/cellular; background transfer subject to OS policy.

- Class: J2ME | Runtime: Java ME / MIDP profile. | Security mechanisms: MIDlet permission model varies by device; application-level protection and limited platform security. | Power controls: Device-specific pause/suspend and low-frequency polling. | Offline storage: Record Management System (RMS), small local queue; encryption only if supported. | Connectivity: GPRS/2G/3G depending on handset; intermittent connection.
- Class: WML | Runtime: Browser-based WML rendering; no full native app runtime. | Security mechanisms: Server-side authentication and HTTPS if supported; very limited local protection. | Power controls: Browser/device-managed power behaviour; no reliable app scheduler assumed. | Offline storage: Prefer no persistent sensitive local storage; temporary minimal form state only. | Connectivity: Feature-phone browser data connection; low bandwidth.

6.4 Information retained for safe synchronisation

- Field: Record ID (globally unique) | Purpose: Prevents duplicate creation when a record is retried.
- Field: Device ID and worker ID | Purpose: Identifies origin and responsible field worker.
- Field: Patient reference | Purpose: Links record to the correct patient without unnecessarily duplicating identity data.
- Field: Creation timestamp and timezone | Purpose: Preserves event order and audit timing.
- Field: Record version / schema version | Purpose: Supports compatibility and conflict detection.
- Field: Payload hash | Purpose: Detects corruption or accidental modification during transfer.
- Field: Sync status | Purpose: Distinguishes pending, in-progress, acknowledged and failed records.
- Field: Retry count and last attempt | Purpose: Controls backoff and helps diagnose repeated failure.
- Field: Server acknowledgement ID | Purpose: Confirms that the server committed the specific record.
- Field: Encryption/key version | Purpose: Allows correct decryption and key rotation management.

6.5 Stage 4 – Complete solution specification

Layered architecture

```
FIELD DEVICE
  Presentation Layer
    - Patient form, validation and worker identity
  Domain Layer
    - Record creation, unique ID, validation, sync state machine
  Data Layer
    - Encrypted local queue, transaction log, acknowledgement store
  Security Layer
    - Device-bound key, encryption, access control
  OS Integration
    - Android WorkManager / iOS background task / J2ME timer
    - Network state, battery state, secure storage
          |
       HTTPS / TLS
          |
SERVER
```

```
API Gateway -> Authentication -> Validation
    -> Idempotency check -> Transactional database
    -> Acknowledgement -> Audit log
```

Data capture and offline storage

7. Authenticate the field worker and load only the minimum patient data needed for the task.
8. Validate every form locally before committing it.
9. Generate a globally unique record ID and creation timestamp.
10. Encrypt the record before writing it to the local queue. Store encryption keys in platform secure storage where available.
11. Commit the record and sync state in one local database transaction.
12. Display a clear pending-sync indicator without exposing patient details on lock-screen notifications.

Security policy

- Android: app-private database; encrypt sensitive fields with an AES-GCM data key protected by Android Keystore; require device authentication where policy demands it.
- iOS: use Keychain for secrets and protected files/database; enable suitable Data Protection class; use TLS with server authentication.
- J2ME: minimise local patient data; apply application-level encryption only if secure key storage and cryptographic support are available; otherwise restrict offline content.
- WML: avoid persistent sensitive local storage; use authenticated HTTPS and short-lived server sessions.
- Lost device: revoke device credentials, disable future API access, trigger remote wipe for managed smartphones where available, rotate tokens and audit the last successful sync.

Ten-hour power policy

- Condition: Normal battery, moving | Capture policy: Capture at configured operational interval; timestamp sensor/location samples. | Sync policy: Batch records and upload when network is available.
- Condition: Stationary | Capture policy: Reduce redundant location sampling; keep user-entered records enabled. | Sync policy: Upload pending records on a scheduled network window.
- Condition: Low battery | Capture policy: Increase non-critical sampling interval; preserve manual clinical records. | Sync policy: Defer bulk sync; permit urgent records.
- Condition: No connectivity | Capture policy: Continue encrypted local queue; do not spin continuously on network retries. | Sync policy: Use exponential backoff and retry when connectivity returns.
- Condition: Charging / Wi-Fi | Capture policy: Normal or higher sampling if required by task. | Sync policy: Prefer bulk synchronisation and verify acknowledgements.

Synchronisation procedure

13. Check connectivity, authentication token validity, battery and sync eligibility.
14. Read a bounded batch of pending records in creation order.
15. Send each record with its unique ID, schema version and payload hash over TLS.
16. Server authenticates the worker/device and validates the payload.

17. Server performs an idempotency check on the unique record ID; a repeated ID returns the previous acknowledgement rather than creating a duplicate.
18. Server commits the record transactionally and returns an acknowledgement ID.
19. Client verifies the acknowledgement and marks only that record as synced.
20. If transfer is interrupted, keep unacknowledged records pending and retry with exponential backoff.
21. On restart, resume from the pending queue; never infer success from a request being sent.
22. Retain audit metadata according to organisational retention policy.

Failure handling

- Failure: App process killed | Handling: Persist every record before showing completion; resume pending queue after restart.
- Failure: Device loses network | Handling: Keep encrypted pending records; retry with bounded exponential backoff.
- Failure: Server receives duplicate request | Handling: Idempotency key prevents duplicate database insertion.
- Failure: Corrupt payload | Handling: Hash validation rejects payload; keep local copy and flag for retry/repair.
- Failure: Lost Android/iPhone | Handling: Revoke device token, use remote management wipe where enrolled, rotate credentials.
- Failure: Feature phone lacks secure storage | Handling: Minimise local data; use short sessions and avoid retaining full patient records; restrict sensitive offline workflow.
- Failure: Storage full | Handling: Stop accepting new records before overwriting unsynced data; alert worker and request sync or managed storage recovery.

6.6 Stage 5 – Desk-check: Class A offline six hours, battery 20%, then lost

- Step: 1 | State / decision: Worker signs in; app obtains protected key and starts encrypted offline queue. | Data exposure and recovery: No plaintext patient database is exposed to other ordinary apps.
- Step: 2 | State / decision: Six hours offline; records are encrypted and marked PENDING. | Data exposure and recovery: Records remain locally available to the authorised app; server has not yet received them.
- Step: 3 | State / decision: Battery reaches 20%; policy reduces non-critical sampling and preserves manual records. | Data exposure and recovery: Queued data remains encrypted; no record is discarded.
- Step: 4 | State / decision: Device is lost before sync; no server acknowledgement exists for pending records. | Data exposure and recovery: Server cannot recover records that were never uploaded.
- Step: 5 | State / decision: Administrator revokes device token and triggers remote lock/wipe if device management is enabled. | Data exposure and recovery: Future access is blocked; remote wipe depends on device connectivity and management enrolment.
- Step: 6 | State / decision: If device later reconnects before wipe, it may be remotely locked/wiped; otherwise local unsynced records may be unrecoverable. | Data exposure and recovery: Encrypted data is protected at rest, but unsynced records may be lost permanently.

Exposure assessment: with a locked, encrypted, non-rooted Android device and hardware-protected key, ordinary theft does not directly reveal the database. Residual exposure remains if the device was

unlocked, credentials were compromised, the OS was rooted or keys were accessible to an attacker. Data loss: records not yet synchronised may be lost if the device cannot be recovered. Recoverable data: previously acknowledged server records remain available from the server.

6.7 Stage 6 – Adversarial tests

- Adverse condition: Stolen unlocked phone | Prediction: Attacker may access the active app and visible patient information. | Result and refinement: Require app re-authentication for sensitive actions, short auto-lock, protected notifications and remote revocation. Residual risk remains until lock/wipe.
- Adverse condition: Sync interrupted after server commit but before client acknowledgement | Prediction: Client retries the record; duplicate risk exists without idempotency. | Result and refinement: Server unique record ID returns existing acknowledgement; client marks synced after receiving it. Design survives if idempotency is transactional.
- Adverse condition: Class C handset has no secure storage | Prediction: Local patient data may be exposed if stored persistently. | Result and refinement: Do not cache full patient records; use minimum data, short session, server-side workflow or restrict offline functionality. Original full offline requirement cannot be guaranteed on this device.
- Adverse condition: Device storage becomes full | Prediction: New records may fail or unsynced records may be overwritten. | Result and refinement: Reserve space, monitor queue size, never overwrite pending records and alert worker before capture fails.
- Adverse condition: Clock changes on device | Prediction: Ordering and audit timestamps may become unreliable. | Result and refinement: Use server receipt time plus device timestamp and monotonic sequence number.

6.8 Final evaluation

A single native application cannot serve all three classes with equal security and offline capability. Android and iOS can support encrypted offline databases and managed synchronisation. J2ME can support a restricted offline queue depending on device security. WML-only devices cannot be assumed to provide secure persistent offline storage. Therefore, the platform strategy must explicitly limit Class C data retention or provide a managed alternative device. Reliable no-duplicate synchronisation depends on unique record IDs, server-side idempotency, transactional commits and explicit acknowledgements.

PART II – MOBILE SYSTEMS ENGINEERING

Part II requires practical implementation and measured evidence. The following sections provide an implementation plan and analysis structure. Commands and test procedures are reproducible examples; the student must run them and paste actual output and screenshots into the final submission. Do not present the predicted behaviour below as a measured result.

TASK 1 – ANDROID VS iOS RESOURCE-MANAGEMENT MODEL

1.1 Layer-by-layer responsibility

- Layer: Kernel / core OS | Android: Linux kernel handles scheduling, memory, drivers, networking, Binder support and SELinux. | iOS: Darwin/XNU provides Mach and BSD services, scheduling, virtual memory, networking and device interfaces.
- Layer: Hardware interface | Android: HAL connects framework/native services to vendor hardware implementation. | iOS: IOKit and platform drivers connect Darwin to hardware.
- Layer: Runtime | Android: ART manages DEX execution, JIT/AOT compilation, class loading and managed heap. | iOS: Objective-C runtime handles dynamic dispatch and metadata; Swift/Objective-C objects use ARC.
- Layer: Framework | Android: Android framework managers expose activities, services, notifications and content APIs. | iOS: Cocoa Touch/UIKit exposes application lifecycle, views, notifications and app APIs.
- Layer: Process creation | Android: Zygote forks app processes with app identity and runtime setup. | iOS: launchd coordinates process launch and system services; app launch activates app process.
- Layer: IPC | Android: Binder driver and framework Binder interfaces. | iOS: Mach ports and XPC.

1.2 Suggested Android evidence commands

```
adb shell ps -A
adb shell dumpsys activity processes
adb shell dumpsys meminfo <package.name>
adb shell cat /proc/<PID>/oom_score_adj
adb shell dumpsys package <package.name>
adb logcat -v time
```

Interpretation: ps shows process identity and state; dumpsys activity processes shows Activity Manager process importance; dumpsys meminfo reports memory categories and PSS; oom_score_adj is an input to Linux low-memory process selection, where a higher adjustment generally indicates a process is more eligible for killing. Exact output varies by Android release and vendor.

1.3 Lifecycle and memory-condition test matrix

- Condition: Foreground | Prediction: Process should remain active while user is interacting; it has high importance. | Evidence to capture: PID, process state, Activity lifecycle logs and PSS.
- Condition: Background for five minutes | Prediction: Process may remain cached, receive limited background execution or be stopped depending on memory and OS policy. | Evidence to capture: dumpsys activity processes, oom_score_adj, logcat and process list.

- Condition: Low-memory pressure | Prediction: Cached/background process may be trimmed or killed; foreground process is generally protected more strongly. | Evidence to capture: Before/after PID, trim callbacks, process death and state restoration logs.

1.4 Example instrumentation and interpretation

Add lifecycle logging to a test application for onCreate, onStart, onResume, onPause, onStop, onDestroy and onSaveInstanceState. Record a timestamp and process ID for each event. A process killed by the OS does not necessarily receive onDestroy. Therefore, persistent state must be saved before process death rather than relying on a final callback.

1.5 Boundary test

Rotate the screen while a form contains unsaved text and a background request is active. This tests Activity recreation, state restoration and cancellation/reconnection of asynchronous work. Expected safe behaviour: form state is restored, duplicate network requests are prevented and the UI does not hold an Activity reference from a long-lived worker.

iOS live evidence requires a macOS/Xcode environment and an iOS simulator or physical device. Since the supplied brief does not provide an iOS device log, no live iOS command output is claimed.

TASK 2 – ARCHITECTURE ASSUMPTIONS VS MEASURED REALITY

2.1 Two implementations

- Solution: A – Single-threaded | Design: Run CPU-heavy parsing/filtering on the main thread. | Purpose: Establish baseline and demonstrate UI blocking risk.
- Solution: B – Background executor | Design: Run the same computation on a worker executor/coroutine; publish results to UI thread. | Purpose: Keep UI responsive while work executes separately.

2.2 Workload and measurement plan

- Use the same input dataset and algorithm for both solutions.
- Warm up the code before recording repeated runs.
- Record wall-clock duration, CPU time, thread count, context switches, heap/PSS, allocations and frame timing.
- Use Android Studio Profiler or Perfetto for thread scheduling and CPU placement evidence.
- Use frame timing / JankStats or FrameTimeline to observe dropped frames.
- Repeat each case at least five times and report median and range.

2.3 Example command and profiler evidence

```
adb shell top -H -p <PID>
adb shell dumpsys meminfo <package.name>
adb shell dumpsys gfxinfo <package.name> framestats
# Capture a Perfetto trace using Android Studio or the Perfetto UI.
```

2.4 Counter-example: more threads can be slower

Use a very small computation, such as summing a short array, and compare one-thread execution with creating a new thread for every task. For small work, thread creation, scheduling, synchronisation and result handoff may cost more than the computation itself. A second counter-example is a shared lock around the critical section: many threads contend for one lock and execute serially while paying context-switch overhead.

- Metric: Task duration | Single-threaded: Record actual median | Multi-threaded: Record actual median | Actual result: Record after test
- Metric: CPU time | Single-threaded: Record actual | Multi-threaded: Record actual | Actual result: Record after test
- Metric: Thread count | Single-threaded: Record actual | Multi-threaded: Record actual | Actual result: Record after test
- Metric: PSS / heap | Single-threaded: Record actual | Multi-threaded: Record actual | Actual result: Record after test
- Metric: Dropped frames | Single-threaded: Record actual | Multi-threaded: Record actual | Actual result: Record after test
- Metric: Energy estimate | Single-threaded: Record actual | Multi-threaded: Record actual | Actual result: Record after test

2.5 Identify first degrading decision

The first degrading decision in the tiny-workload case is creating a worker thread for a task whose computation is cheaper than the thread creation and coordination overhead. In a lock-contention case, the first harmful decision is placing shared mutable work behind a single heavily contended lock. The remedy is a persistent bounded executor, batching, reduced shared state and avoiding unnecessary parallelism.

2.6 Changed requirement: full working day

When battery life is more important than minimum latency, use a bounded worker pool, batch parsing and sensor processing, defer non-urgent work through WorkManager and allow efficiency cores to execute background tasks when the scheduler chooses. Avoid busy loops and repeated thread creation. Keep urgent tracking events separate from bulk processing. The final choice must be justified using measured CPU time, wakeups, energy estimate and responsiveness from the two implementations.

TASK 3 – RUNTIME BEHAVIOUR UNDER CHANGING CONDITIONS

3.1 Benchmark workloads

- Workload: CPU-bound loop / algorithm | What it measures: Steady-state arithmetic and branch execution. | Expected runtime sensitivity: JIT/AOT compilation should affect hot repeated code.
- Workload: Object-allocation-heavy routine | What it measures: Allocation rate, heap growth and garbage collection. | Expected runtime sensitivity: GC frequency and pause duration are key.
- Workload: UI-list rendering | What it measures: Frame time, scrolling smoothness and allocation during binding. | Expected runtime sensitivity: Layout, object churn, main-thread work and rendering pipeline.

3.2 ART compilation and evidence commands

```
adb shell dumpsys package <package.name>
adb shell cmd package compile -m verify -f <package.name>
adb shell cmd package compile -m speed-profile -f <package.name>
adb shell cmd package compile -m speed -f <package.name>
adb logcat -v time | grep -i -E "art|gc|heap"
adb shell dumpsys meminfo <package.name>
```

Compilation filter names and availability can vary with Android version. Capture the exact command output and OS build. A verify-oriented mode may reduce ahead-of-time optimisation; speed-profile uses profile information; speed requests broader ahead-of-time compilation and can increase storage use. The CPU-heavy hot loop is expected to show the clearest steady-state difference.

3.3 Prediction table before changing compilation mode

- Mode: Verify / interpret-oriented | Prediction for CPU workload: Lower steady-state speed for hot code if more interpretation is used. | Startup / storage expectation: Potentially less compilation work/storage; startup may be lighter but runtime work can increase.
- Mode: Speed-profile | Prediction for CPU workload: Frequently executed paths receive profile-guided optimisation. | Startup / storage expectation: Balanced startup and steady-state performance; profile data affects result.
- Mode: Speed | Prediction for CPU workload: More code compiled ahead of time. | Startup / storage expectation: May improve steady-state execution but increase compilation time and storage footprint.

3.4 GC and heap evidence

Record heap size before and after the allocation workload, GC event count, GC pause duration and application frame timing. Increasing heap usage without returning toward a stable range may indicate a leak or unbounded cache. Repeated short GC cycles with poor progress may indicate allocation pressure or thrashing.

3.5 Production failure injection

- Memory leak: retain an Activity through a static/global reference or grow a cache without an eviction policy.

- Main-thread blocking: perform a deliberately slow operation on the UI thread in a controlled test build.
- Detection: capture heap growth, LeakCanary report, StrictMode violation, ANR trace or OutOfMemoryError.
- Correction: remove the retained reference, bound the cache, move blocking work to a worker executor and publish only results to the UI thread.

3.6 Cross-platform runtime comparison

- Runtime: Native Android ART | Execution and memory: Kotlin/Java DEX; ART uses interpretation, JIT and profile-guided AOT; tracing GC. | UI thread and native APIs: UI updates on main thread; JNI connects to native code; Android framework exposes platform APIs. | Trade-off: Strong Android integration; GC and runtime overhead must be measured.
- Runtime: Native iOS Swift / ARC | Execution and memory: Native compiled code; ARC retains/releases class objects. | UI thread and native APIs: UIKit work on main thread; Swift/Objective-C can call system frameworks directly. | Trade-off: Strong iOS integration and no tracing GC pause, but reference cycles can leak.
- Runtime: Flutter / Dart | Execution and memory: Dart can use AOT compilation in release builds; managed heap and Dart GC. | UI thread and native APIs: Flutter engine renders its UI; platform channels/plugins bridge to native APIs. | Trade-off: Consistent UI and shared code; engine adds footprint and native integration needs plugins.
- Runtime: React Native / WebView hybrid | Execution and memory: JavaScript engine executes JS; bridge or JSI/native modules connect to platform APIs; WebView adds browser runtime. | UI thread and native APIs: UI work must be coordinated with native main thread; native modules provide device APIs. | Trade-off: Fast shared development, but JS/native coordination and engine memory can affect latency and footprint.

Recommendation: choose native applications if deep platform integration, predictable background behaviour and access to secure platform APIs dominate. Flutter is a reasonable single-codebase candidate when consistent UI and shared business logic matter and the measured engine footprint is acceptable. React Native/WebView should be selected only after measuring JS execution, bridge/native-call overhead and memory on target devices.

The assignment asks for a recommendation based on Stage A–C evidence. Since no benchmark has been run in this document, the final selection is provisional and must be confirmed with the student's actual results.

TASK 4 – SECURE MOBILE APPLICATION AUDIT

4.1 Audit input model

- Input: Manifest / Info.plist permissions | Audit checks: Excessive permissions, missing runtime consent, sensitive capabilities.
- Input: Data stores | Audit checks: Plaintext secrets, public files, unencrypted databases, weak key handling.
- Input: Network endpoints | Audit checks: HTTP use, TLS validation, certificate trust, endpoint exposure.
- Input: Exported components / URL schemes | Audit checks: Unprotected exported activities, services, receivers, deep links and intent filters.

4.2 STRIDE threat model

Threat	Asset / attack vector	Protection	Residual risk
Spoofing	Stolen authentication token or forged app identity.	App signing, strong authentication, token binding and secure storage.	Compromised device or stolen unlocked session.
Tampering	Modify local patient database or API payload.	Encrypted storage, integrity checks, TLS and server validation.	Rooted device or compromised app process.
Repudiation	Worker denies creating or changing a record.	Server-side audit log, timestamps, authenticated user and record ID.	Compromised credentials or incorrect device clock.
Information disclosure	Read token/database or intercept API traffic.	Sandbox, encryption, Keystore/Keychain and TLS.	OS compromise, screenshots or authorised-user misuse.
Denial of service	Repeated requests, storage exhaustion or resource abuse.	Rate limits, bounded queues, input validation and backoff.	Large-scale network or server outage.
Elevation of privilege	Abuse exported component or overbroad permission.	Non-exported components, signature permissions, least privilege and SELinux sandbox.	Framework or OS vulnerability.

4.3 Asset decision table

- Asset: Auth token | Attack vector: Read app files or extract from logs. | Mechanism applied: Keystore/Keychain-backed encryption; no token logging. | Result: Partially blocked | Reason: Protects at-rest data, but compromised active process can still use token.
- Asset: Local DB | Attack vector: Copy database from shared storage. | Mechanism applied: App sandbox, encrypted DB and key protected by secure storage. | Result: Partially blocked | Reason: Strong on normal devices; rooted/compromised OS remains residual risk.
- Asset: API traffic | Attack vector: Network interception / MITM. | Mechanism applied: TLS certificate validation; pinning if justified. | Result: Blocked under valid trust assumptions | Reason: TLS protects confidentiality and integrity against ordinary network interception.
- Asset: Exported activity | Attack vector: Malicious app sends crafted intent. | Mechanism applied: Set exported=false unless required; validate caller and input. | Result: Blocked | Reason: Untrusted apps cannot directly invoke a non-exported component.

4.4 Predict-before-execute experiment

Weaken one control by changing an internal component to android:exported="true" without authentication. Prediction: the component becomes reachable to other applications; a malicious app may invoke it with crafted extras; the audit verdict for that component changes from Blocked to Exposed or Partially Blocked. Run the test only on a controlled test application.

```
adb shell am start -n <package.name>/<component.name>
adb logcat -v time
adb shell dumpsys package <package.name>
```

Record the manifest before and after, the command result, logcat output and audit verdict. Restore exported=false or add strict caller authentication and input validation after the test.

4.5 Third-party library leaks device identifier

Asset: device identifier and associated user/device metadata. Affected layer: application networking and transport-security boundary. The sandbox and permission system cannot stop a library inside the trusted app process from transmitting data that the app itself can access. Detection can occur through static code review, dependency/SBOM analysis, network capture, proxy inspection in a test environment or runtime instrumentation. Remediation is to remove or update the library, prohibit identifier collection unless essential, enforce TLS, minimise identifiers and add egress tests. TLS alone does not prevent a library from intentionally sending data to an unauthorised endpoint; endpoint allowlisting and data minimisation are also required.

4.6 Biometric login and BYOD extension

- Add biometric authentication as a local user-verification factor bound to a Keystore/Keychain-protected cryptographic operation; do not store biometric templates in the app.
- Extend the threat model to include biometric spoofing, device passcode fallback, biometric enrollment changes and compromised user sessions.
- Use key attestation where supported to assess whether cryptographic keys are hardware-backed and device integrity requirements are met.
- Separate work and personal data using managed work profiles on Android or managed app/data controls on iOS.
- Define remote wipe scope precisely: remove enterprise data and credentials without assuming personal data may be erased.
- Update audit logic to inspect profile boundaries, managed configuration, key protection, remote revocation and data export paths.

TASK 5 – THREE POWER ENGINEERS, ONE DEVICE

5.1 Technique comparison

- Technique: DVFS / CPU governor | Trigger and state: CPU load, thermal state and power policy. | Information maintained: CPU utilisation, frequency, voltage, core type and temperature. | Expected effect: Lower frequency/voltage at low load reduces CPU energy; high-performance cores boost short workloads. | Side effect: Slower completion under low-power policy; thermal limits may reduce sustained performance.
- Technique: Doze / App Standby | Trigger and state: Screen off, stationary device and inactivity; app bucket policy. | Information maintained: Screen/motion state, idle state, app usage bucket and maintenance windows. | Expected effect: Defers background jobs/network access and reduces wakeups during idle. | Side effect: Delayed sync, notifications or background updates.
- Technique: Wakelock discipline and batching | Trigger and state: Application requests bounded CPU wake or schedules batch work. | Information maintained: Wake-lock duration, job constraints, pending data, network and charging state. | Expected effect: Fewer repeated CPU/radio wakeups; work grouped into fewer sessions. | Side effect: Data delivery latency increases; long wake lock defeats idle benefits.

5.2 Instrumentation

```
adb shell dumpsys batterystats --reset
# Run one controlled test case.
adb shell dumpsys batterystats
adb shell dumpsys deviceidle
adb shell dumpsys jobscheduler
adb shell dumpsys power
```

Battery Historian or Android Studio Energy Profiler can be used to inspect wakelock duration, radio-active time, job scheduling and estimated energy. Record device model, Android build, battery state, screen state, network type, test duration and workload. Compare repeated runs under the same conditions.

5.3 Conflict test

Conflict: hold a partial wakelock continuously while expecting Doze to reduce background activity. Prediction: the wakelock prevents or delays deep sleep, so Doze cannot achieve its intended CPU-idle benefit. A second conflict is scheduling frequent exact alarms that repeatedly wake the device and defeat batching. The first design decision causing the conflict is the application's unnecessary continuous wake request or high-frequency alarm policy.

5.4 Deployment decision

- Environment: A – Always-on location/sensor tracking | Recommended emphasis: Bounded foreground service only when justified, sensor batching, adaptive sampling and carefully scoped wake locks. | Reason: Continuous location may be required, but sensor/radio batching and adaptive intervals reduce fixed wakeup cost.
- Environment: B – Media/gaming-intensive | Recommended emphasis: DVFS and thermal-aware scheduling; keep rendering responsive and avoid background tracking overhead. | Reason: Sustained CPU/GPU/display load dominates energy; aggressive background batching has less impact on foreground render loop.

- Environment: C – Large managed fleet | Recommended emphasis: WorkManager/JobScheduler constraints, managed policies, OS-version compatibility and fleet telemetry. | Reason: A common policy must tolerate different OS versions, battery states and enterprise restrictions.

A technique at one layer can be defeated by another. For example, kernel/OS Doze may defer background work, but an application that continuously holds a wakelock or schedules frequent alarms can keep waking the device. Power management must therefore be coordinated across hardware, kernel, OS scheduler and application behaviour.

TASK 6 – BATTERY-AWARE, SECURE FIELD-TRACKING PROTOTYPE

6.1 Problem model

- Element: Input | Definition: Battery level, charging state, network type, last upload time, queued records, motion state, device temperature and emergency flag.
- Element: Output | Definition: Sampling interval, upload action, queue state, encryption status and estimated energy per cycle.
- Element: Objective | Definition: Maximise useful tracking records delivered per unit of battery energy while protecting data.
- Element: Constraints | Definition: Full shift, changing connectivity, thermal limits, secure storage, OS background restrictions and urgent delivery requirement.

Battery percentage alone is insufficient. Two devices at 20% battery may need different actions: one may be stationary on Wi-Fi and have a large pending queue; another may be moving on LTE during an emergency. Charging state, network cost, motion, queued records, last upload and temperature are needed to choose a safe policy.

6.2 State-based policy

- State dimension: BatteryState | Values / meaning: HIGH (above 50%), NORMAL (16–50%), LOW (15% or below), CHARGING, CRITICAL (for example below 5%).
- State dimension: NetworkState | Values / meaning: WIFI, CELLULAR, OFFLINE.
- State dimension: MotionState | Values / meaning: MOVING, STATIONARY, UNKNOWN / EMERGENCY.

Default state: NORMAL battery, OFFLINE network until detected, UNKNOWN motion. Every policy transition encrypts records before local queueing and requires authenticated TLS for upload.

- Battery: HIGH | Network: WIFI | Motion: MOVING | Sampling policy: Frequent operational interval, e.g. 10 s. | Upload action: Upload queued batch.
- Battery: HIGH | Network: CELLULAR | Motion: MOVING | Sampling policy: Normal interval, e.g. 15 s. | Upload action: Upload bounded batch; avoid repeated connection setup.
- Battery: NORMAL | Network: WIFI | Motion: STATIONARY | Sampling policy: Moderate interval, e.g. 60 s. | Upload action: Upload queued batch.
- Battery: LOW | Network: CELLULAR | Motion: STATIONARY | Sampling policy: Reduced interval, e.g. 120 s. | Upload action: Defer bulk upload unless queue age limit reached.
- Battery: ANY | Network: OFFLINE | Motion: ANY | Sampling policy: Use motion-adaptive sampling; timestamp each record. | Upload action: Queue encrypted data; no rapid retry loop.
- Battery: CHARGING | Network: WIFI | Motion: ANY | Sampling policy: Operational sampling interval. | Upload action: Prefer full synchronisation.
- Battery: ANY | Network: ANY | Motion: EMERGENCY | Sampling policy: Emergency interval, e.g. 2–5 s if permitted. | Upload action: Prioritise authenticated immediate upload.

6.3 Layered architecture

```
UI (Main Thread)
  - Map, status, battery and sync indicator
Domain (Background Executor)
  - Policy Engine / Sampling Controller
  - Record Validator / Sync State Machine
Data Layer
  - Encrypted local queue / transaction store
Security Layer
  - AES-GCM record encryption
  - Android Keystore protected key
OS Services
  - Location / Sensor APIs
  - Foreground service when user-visible continuous tracking is required
  - WorkManager for deferrable batch sync
Network
  - TLS authenticated API / idempotent server endpoint
```

6.4 Six-condition scenario table

- Condition: Charging + Wi-Fi + moving | Policy: NORMAL / charging | Sampling / upload: 10–15 s; upload batch immediately. | Security: AES-GCM before queue; TLS upload. | Energy estimate: Record actual or model estimate.
- Condition: 80% + LTE + moving | Policy: HIGH | Sampling / upload: 15 s; bounded cellular upload. | Security: Encrypted queue; TLS. | Energy estimate: Record actual or model estimate.
- Condition: 20% + stationary + LTE | Policy: LOW | Sampling / upload: 120 s; defer bulk upload to Wi-Fi/charging window. | Security: Encrypted queue; no plaintext fallback. | Energy estimate: Record actual or model estimate.
- Condition: Low-power mode + Wi-Fi | Policy: LOW | Sampling / upload: 120 s; scheduled batch upload. | Security: Encrypted queue; TLS. | Energy estimate: Record actual or model estimate.
- Condition: No connectivity + moving | Policy: Any / OFFLINE | Sampling / upload: Motion-based sampling; queue locally; no repeated polling. | Security: Encrypt before local commit. | Energy estimate: Record actual or model estimate.
- Condition: Device hot + cellular | Policy: Thermal constrained | Sampling / upload: Reduce non-urgent sampling; defer bulk work. | Security: Encryption remains mandatory; TLS when sending. | Energy estimate: Record actual or model estimate.

The example sampling intervals are design parameters for the prototype, not measured optimum values. Estimate energy per cycle only after defining a power model or measuring with profiler/Battery Historian. Do not enter fabricated mAh figures.

6.5 Prototype policy pseudocode

```
ALGORITHM ChooseTrackingPolicy(Battery, Charging, Network,
                               Motion, Temperature, Emergency)

BEGIN
    IF Emergency = TRUE THEN
        SamplingInterval <- 5 seconds
        UploadAction <- PRIORITY_UPLOAD
    ELSE IF Temperature = HOT THEN
```

```
        SamplingInterval <- 120 seconds
        UploadAction <- DEFER_BULK
    ELSE IF Network = OFFLINE THEN
        IF Motion = MOVING THEN
            SamplingInterval <- 30 seconds
        ELSE
            SamplingInterval <- 120 seconds
        END IF
        UploadAction <- ENCRYPT_AND_QUEUE
    ELSE IF Charging = TRUE AND Network = WIFI THEN
        SamplingInterval <- 15 seconds
        UploadAction <- SYNC_PENDING_BATCH
    ELSE IF Battery <= 15 AND Motion = STATIONARY THEN
        SamplingInterval <- 120 seconds
        UploadAction <- DEFER_BULK
    ELSE IF Battery <= 15 THEN
        SamplingInterval <- 60 seconds
        UploadAction <- BATCH_WHEN_ALLOWED
    ELSE
        SamplingInterval <- 15 seconds
        UploadAction <- BATCH_UPLOAD
    END IF

    ENCRYPT record using AES-GCM before local storage
    IF UploadAction permits transfer THEN
        AUTHENTICATE server and use TLS
        SEND idempotent batch
        MARK only acknowledged records as synced
    END IF

    RETURN SamplingInterval, UploadAction
END
```

6.6 Validation against an always-on baseline

- Scenario: High battery, Wi-Fi, moving | Always-on baseline: Frequent sampling and immediate upload. | Adaptive policy expectation: Normal sampling and batched upload. | Comparison to complete: Record actual sample count, wakeups and energy.
- Scenario: Low battery, stationary, LTE | Always-on baseline: Frequent sampling and cellular radio wakeups. | Adaptive policy expectation: Longer interval and deferred bulk upload. | Comparison to complete: Record actual sample count, radio-active time and energy.
- Scenario: Offline, moving | Always-on baseline: Frequent sample attempts and failed network retries if poorly designed. | Adaptive policy expectation: Queue encrypted records; no network polling loop. | Comparison to complete: Record queue completeness, wakeups and energy.

Independent validation: manually calculate the expected number of samples from shift duration divided by sampling interval, then compare with prototype output. For example, a 60-minute stationary period at 120-second intervals should generate approximately 30 samples, excluding boundary and scheduling jitter. Compare record IDs and queue counts to verify that the implementation matches the policy.

6.7 Scalability and load

- Load: 100 records | Policy evaluations: Record actual | Wakeups: Record actual | Memory / battery observation: Record actual
- Load: 1,000 records | Policy evaluations: Record actual | Wakeups: Record actual | Memory / battery observation: Record actual
- Load: 10,000 records | Policy evaluations: Record actual | Wakeups: Record actual | Memory / battery observation: Record actual
- Load: 1 / 5 / 10 / 20 devices | Policy evaluations: Record actual | Wakeups: Record actual | Memory / battery observation: Record actual

Batching reduces repeated radio wakeups because the radio has a fixed energy cost to transition into an active state. However, continuous high-frequency tracking still consumes energy through sensor sampling, CPU processing, memory writes and the volume of data transmitted. Batching reduces connection overhead; it does not make continuous sensing free.

6.8 Changed business requirement – 5,000 devices and emergency updates

The requirement changes from energy efficiency over a full shift to low-latency, reliable delivery within a few seconds during emergencies. The existing low-battery sampling and deferred-upload policy becomes unsuitable because it deliberately increases intervals and waits for a preferred network window. Investigate an adaptive priority policy: normal mode remains battery-aware, while emergency mode enables a bounded foreground tracking service, high-priority event delivery, immediate authenticated uploads, server-side fleet coordination and fallback connectivity.

Trade-offs include higher energy consumption, device heat, cellular data cost and greater privacy exposure from more frequent location transmission. Emergency mode should be explicitly activated, audited, time-limited where possible and visible to the user/organisation.

6.9 Technical reflection – under 150 words

No single power-management technique is optimal for every application because workloads, urgency, hardware use and connectivity differ. Batching reduces radio wakeups for offline or low-priority tracking, but it is unsuitable when an emergency requires location updates within seconds. DVFS can reduce CPU energy during light work, while a sustained graphics workload may need higher performance to avoid frame drops. Doze saves energy when the device is idle, but a continuous tracking requirement may need a justified foreground service and carefully bounded wake locks. The correct policy therefore depends on battery state, motion, network, temperature, queued data and business priority. The prototype should measure wakeups, CPU time, radio-active time, queue size and delivery delay before selecting thresholds.

Final submission evidence checklist

- Attach Android/iOS device or emulator details and OS version.
- Attach actual adb command outputs, lifecycle logs and screenshots for Part II Task 1.
- Attach both single-threaded and multithreaded implementations and profiler traces for Task 2.

- Attach ART compilation command output, GC logs, heap evidence and failure detection for Task 3.
- Attach manifest/Info.plist, audit output, attack demonstration and remediation evidence for Task 4.
- Attach Battery Historian / batterystats / profiler traces for each power technique and conflict test for Task 5.
- Attach prototype source, six-condition output, independent validation and load measurements for Task 6.
- Replace every “Record actual” field with real observations; state device, test duration and measurement method.

OVERALL CONCLUSION

Mobile operating systems differ in kernel structure, runtime, application lifecycle, security enforcement and power-management policy. Android uses the Linux kernel, HAL, native libraries, ART and application framework; iOS uses Darwin/XNU, system frameworks and Objective-C/Swift runtime mechanisms; Windows 8 uses the Windows NT kernel and WinRT application model. These differences influence process creation, background delivery, memory management, IPC and native API access.

A secure mobile application must use least-privilege permissions, sandboxed and encrypted storage, trusted code signing and authenticated transport. Battery efficiency requires coordinated decisions across CPU frequency, OS idle controls, application wake locks, sensor batching and network scheduling. For RuralCare, native smartphone applications provide stronger secure offline storage and synchronisation, while legacy feature phones require a restricted workflow where secure local storage cannot be guaranteed.

The integrated field-tracking design uses a state-based policy driven by battery, network, motion, temperature and emergency state. It encrypts records before queueing, synchronises through authenticated transport and uses idempotent server processing to avoid duplication. Final engineering conclusions must be supported by reproducible measurements and boundary tests. As required by the assignment, successful execution alone is not sufficient; the design must also be instrumented, challenged, debugged and validated.

REFERENCES

- ITA03 – Mobile Computing, Assignment 4A and Assignment 4B, supplied assignment brief, pages 1–25.
- Android Developers documentation: Processes and app lifecycle; Android Runtime (ART); app sandbox; permissions; Android Keystore; WorkManager; Doze and App Standby.
- Apple Developer Documentation: App lifecycle; Background execution; Keychain Services; Data Protection; App Sandbox and APNs.
- Microsoft documentation: Windows Runtime (WinRT), Windows app lifecycle and AppContainer security model.

- Android platform tools documentation: adb, dumpsys, logcat, package compilation and batterystats.
- Perfetto documentation: system tracing and performance analysis.